INDEX

SNo	Name of the Experiment	Program (5M)	MCQs (5M)	Execution (5M)	Total (15M)
1	Study of Network devices in detail and connect the computers in Local Area	**5**	**5**	**5**	**15**
2	Data Link Layer Framing Methods — Character Stuffing & Bit Stuffing	**5**	**5**	**5**	**15**
3	Data Link Layer Framing — Checksum	**5**	**5**	**5**	**15**
4	Write a program for Hamming Code generation for error detection and correction.	**5**	**5**	**5**	**15**
5	Write a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.	**5**	**5**	**5**	**15**
6	Write a Program to implement Sliding window protocol for Goback N.	**5**	**5**	**5**	**15**
7	Write a Program to implement Sliding window protocol for Selective repeat.	**5**	**5**	**5**	**15**
8	Write a Program to implement Stop and Wait Protocol.	**5**	**5**	**5**	**15**
9	Write a program for congestion control using leaky bucket algorithm.	**5**	**5**	**5**	**15**
10	Write a Program to implement Dijkstra‘s algorithm to compute the Shortest path through a graph.	**5**	**5**	**5**	**15**
11	Write a Program to implement Distance vector routing algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes)	**5**	**5**	**5**	**15**
12	Write a Program to implement Broadcast tree by taking subnet of hosts	**5**	**5**	**5**	**15**
13	Study about Wireshark	**5**	**5**	**5**	**15**
14	How to run Nmap scan.	**5**	**5**	**5**	**15**
15	Operating System Detection using Nmap	**5**	**5**	**5**	**15**
16	Do the following using NS2 Simulator	**5**	**5**	**5**	**15**

Marks : Average(Total) = 15.0

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:55:31 PM**

Experiment-1

AIM:

The aim of this experiment is to study network devices in detail and to connect computers in a Local Area Network (LAN) using a switch, Ethernet cables, and IP configuration.

PROBLEM STATEMENT:

1. **Study** the different network devices (Hub, Switch, Router, Bridge, Repeater, Gateway).
2. **Connect** at least three computers in a LAN using a switch and Ethernet cables.
3. **Configure** IP addresses, subnet masks, and default gateways on each computer.
4. **Test** connectivity between the computers using the `ping` command.
5. **Document** the IP configuration and connectivity results.

INPUT:

- Computers with Ethernet NICs
- Ethernet switch
- Cat5e/Cat6 Ethernet cables with RJ45 connectors
- (Optional) Router for internet connectivity

OUTPUT:

- A working LAN with three or more connected computers
- IP configuration details for each computer
- Successful `ping` responses between computers

Example Output (Command Prompt):

```
C:\Users\PC1> ipconfig

Ethernet adapter Ethernet0:
   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . . . . . . . : 192.168.1.10
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.1.1

C:\Users\PC1> ping 192.168.1.11

Pinging 192.168.1.11 with 32 bytes of data:
Reply from 192.168.1.11: bytes=32 time<1ms TTL=128
Reply from 192.168.1.11: bytes=32 time<1ms TTL=128
Reply from 192.168.1.11: bytes=32 time<1ms TTL=128
Reply from 192.168.1.11: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.11:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

ALGORITHM / PROCEDURE:

Part A: Study of Network Devices

Step-1: Understand the function of each network device:

- **NIC**: Connects a computer to the network.
- **Hub**: Broadcasts data to all ports (obsolete).

- **Switch**: Forwards data intelligently based on MAC addresses.
- **Router**: Connects different networks based on IP addresses.
- **Bridge**: Connects two network segments.
- **Repeater**: Regenerates signals to extend distance.
- **Gateway**: Translates between different protocols.

Part B: Connect Computers in a LAN

Step-1: Gather all required hardware: computers, Ethernet switch, and Ethernet cables.
Step-2: Connect each computer to the switch using an Ethernet cable (RJ45). Ensure the cable is firmly inserted into both the computer's NIC and the switch's port.
Step-3: Turn on the computers and the switch. Observe the link/activity LEDs on the switch and NICs - they should light up, indicating a physical connection.

Part C: Configure IP Addresses

Step-1: Decide on a private IP address range (e.g., `192.168.1.0/24`).
Step-2: Assign unique IP addresses to each computer:

Computer	IP Address	Subnet Mask	Default Gateway
PC1	192.168.1.10	255.255.255.0	192.168.1.1
PC2	192.168.1.11	255.255.255.0	192.168.1.1
PC3	192.168.1.12	255.255.255.0	192.168.1.1

Step-3: Configure the IP addresses on each computer using the operating system's network settings.

Windows Configuration:
1. Open **Control Panel** -> **Network and Sharing Center** -> **Change adapter settings**.
2. Right-click on **Ethernet** -> **Properties**.
3. Select **Internet Protocol Version 4 (TCP/IPv4)** -> **Properties**.
4. Select **Use the following IP address**.
5. Enter the IP address, subnet mask, and default gateway.
6. Click **OK** to save.

Linux Configuration (Ubuntu):
1. Open a terminal.
2. Edit network configuration: `sudo nano /etc/netplan/01-netcfg.yaml` (or use `nmcli`).
3. Add or edit the network interface configuration.
4. Apply changes: `sudo netplan apply`.

Part D: Test Connectivity

Step-1: Open the Command Prompt (Windows) or Terminal (Linux).
Step-2: Run `ipconfig` (Windows) or `ifconfig` / `ip addr` (Linux) to verify IP configuration.
Step-3: Use `ping` to test connectivity:
- `ping 192.168.1.11` (from PC1 to PC2)
- `ping 192.168.1.12` (from PC1 to PC3)
- `ping 192.168.1.10` (from PC2 to PC1)

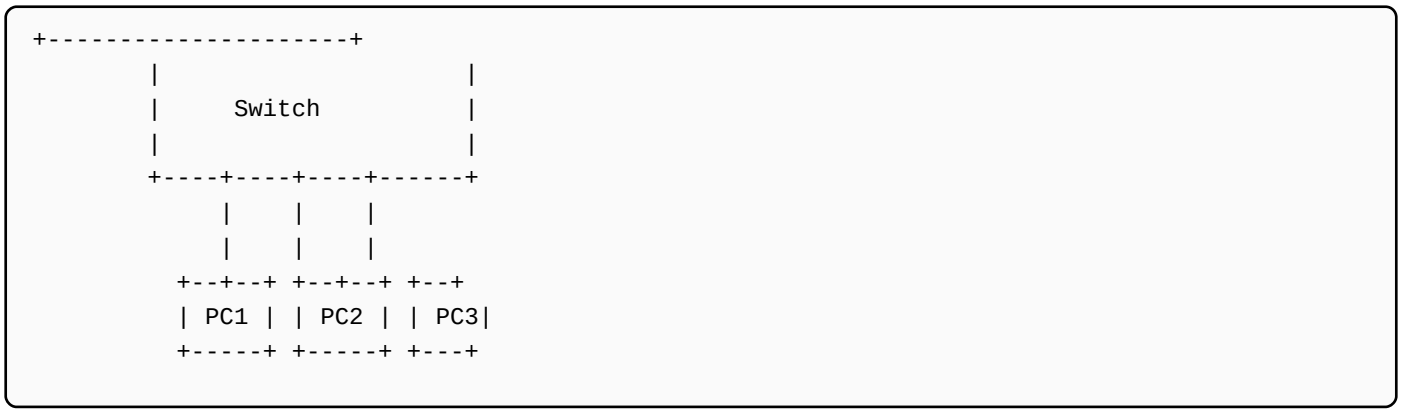
Step-4: Observe the output - successful replies confirm LAN connectivity.

EXAMPLES:

Example 1: Star Topology LAN

- **Devices**: 4 Computers, 1 Switch, 4 Ethernet Cables
- **Configuration**:
  - PC1: 192.168.1.10/24
  - PC2: 192.168.1.11/24
  - PC3: 192.168.1.12/24
  - PC4: 192.168.1.13/24
- **Connectivity**: All computers can ping each other.

Example 2: Network Diagram



CONSTRAINTS:

- All computers must be in the same IP subnet (e.g., `192.168.1.0/24`).
- IP addresses must be unique within the LAN.
- The switch must be powered and functioning.
- Ethernet cables must be properly crimped and tested.

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/satishmettela04-art/exp-1-cn-.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: satishmettela04-art
Repo Name: exp-1-cn-
Default Branch: main
Fetched Files:
```

Performance Summary

Solution evaluation complete

100	0%	0ms
POINTS	RATE	TIME

ACCURACY **0/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 5:03:54 PM**

Experiment-2.1

AIM:

The aim of this program is to implement **character stuffing** in the data link layer, where special characters are stuffed with an escape character to prevent misinterpretation of frame boundaries.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts a string from the user as data.
2. Accepts a position at which a character is to be inserted.
3. Accepts a character to be inserted at that position.
4. Performs character stuffing by:
   - Adding start flag `DLESTX` at the beginning
   - Inserting `DLE` before any `DLE` characters in the data
   - Inserting the given character at the specified position (with `DLE` stuffing around it)
   - Adding end flag `DLEETX` at the end
5. Displays the stuffed frame.

INPUT:

- A string (data)
- A position (integer) where a character is to be inserted
- A character to insert at that position

OUTPUT:

- The stuffed frame with flags and escape characters

Example Output:

```
Enter string: necnrt
Enter position: 2
Frame after stuffing:
DLESTX n DLE c DLE e DLE c DLE n DLE r DLE t DLEETX
```

ALGORITHM:

Step-1: Start the program.
Step-2: Accept the input string from the user.
Step-3: Accept the position where the character is to be inserted.
Step-4: Accept the character to be inserted.
Step-5: Check if the position is valid (within the string length). If not, ask again.
Step-6: Initialise the stuffed frame with the start flag `DLESTX`.
Step-7: For each character in the input string:
   - If the character is `DLE` (escape character), insert an extra `DLE` before it.
   - If the current position is the insertion position, insert the character with `DLE` stuffing.
Step-8: Add the end flag `DLEETX` at the end.
Step-9: Display the stuffed frame.
Step-10: End.

EXAMPLES:

Example 1: Basic Character Stuffing

- **Input:** `necnrt`, Position: `2`, Character: `c`
- **Output:** `DLESTX n DLE c DLE e DLE c DLE n DLE r DLE t DLEETX`

Example 2: Character Stuffing with DLE in Data

- **Input:** `A B C DLE D E`, Position: `1`, Character: `X`
- **Output:** `DLESTX DLE X DLE A B C DLE DLE D E DLEETX`

CONSTRAINTS:

- The string must not contain the escape character `DLE` without proper handling.
- The position must be within the length of the string.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#include <string.h>
int main()
{
 char a[100], b[200], ch;
 int pos, i, j = 0;
 scanf("%s", a);
 scanf("%d", &pos);
 scanf(" %c", &ch);
 strcpy(b, "DLESTX");
 j = strlen(b);
 for(i = 0; a[i] != '\0'; i++)
 {
 if(i == pos - 1)
 {
 b[j++] = 'D';
 b[j++] = 'L';
 b[j++] = 'E';
 b[j++] = ch;
 b[j++] = 'D';
 b[j++] = 'L';
 b[j++] = 'E';
 }
 b[j++] = a[i];
 }
 b[j] = '\0';
 strcat(b, "DLEETX");
 printf("stuffed Frame: %s",b);

 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT	EXPECTED
`["ABCDEF",3,"X"]`	`Stuffed Frame: DLESTXABDLEXDLECDEFDLEETX`

ACTUAL OUTPUT

```
stuffed Frame: DLESTXABDLEXDLECDEFDLEETX
```

TEST #2 **PASSED**

INPUT
```
["HELLO",2,"A"]
```

EXPECTED
```
Stuffed Frame: DLESTXHDLEADLEELLODLEETX
```

ACTUAL OUTPUT
```
stuffed Frame: DLESTXHDLEADLEELLODLEETX
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **2/2 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 5:03:54 PM**

Experiment-2.2

AIM:

The aim of this program is to implement **bit stuffing** in the data link layer, where a `0` bit is inserted after every occurrence of five consecutive `1`s in the data to prevent false flag detection.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts a binary string (consisting only of `0`s and `1`s) from the user.
2. Performs bit stuffing by:
   - Scanning the data from left to right.
   - Whenever five consecutive `1`s are found **and** the next bit is `1`, inserting a `0` after those five `1`s.
3. Displays the stuffed bit string.

> **Note:** The program does **not** add start/end flags; it only performs the stuffing. This matches the typical lab exercise for bit stuffing.

INPUT:

- A binary string (e.g., `0101111101011111110101111111`)

OUTPUT:

- The stuffed binary string after inserting `0`s where needed.

Example Output:

```
Enter your data: 0101111101011111110101111111
Your stuffing data is: 01011111010111111010101111111011
```

ALGORITHM:

Step-1: Start the program.
Step-2: Accept the input binary string from the user.
Step-3: Initialize an empty string for the stuffed data.
Step-4: Initialize a counter `count` to `0` to track consecutive `1`s.
Step-5: For each character in the input string:
- Append the character to the stuffed string.
- If the character is `'1'`, increment `count`.
- Else (character is `'0'`), set `count` to `0`.
- If `count` reaches `5` **and** the next character (if any) is `'1'`, append a `'0'` to the stuffed string and reset `count` to `0`.

Step-6: Display the stuffed string.
Step-7: End.

EXAMPLES:

Example 1: Basic Bit Stuffing
- **Input:** `0101111101011111110101111111`
- **Output:** `01011111010111111010101111111011`

Example 2: No Stuffing Needed
- **Input:** `01010101`
- **Output:** `01010101` (no five consecutive `1`s)

Example 3: Stuffing at the End
- **Input:** `111111`
- **Output:** `1111101` (after five `1`s, a `0` is inserted, then the remaining `1`)

CONSTRAINTS:

- The input must contain only characters `'0'` and `'1'`.
- The program does not handle non-binary input (may be added as an extension).

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#include <string.h>
int main()
{
 char a[100], b[200], ch;
 int pos, i, j = 0;
 scanf("%s", a);
 scanf("%d", &pos);
 scanf(" %c", &ch);
 strcpy(b, "DLESTX");
 j = strlen(b);
 for(i = 0; a[i] != '\0'; i++)
 {
 if(i == pos - 1)
 {
 b[j++] = 'D';
 b[j++] = 'L';
 b[j++] = 'E';
 b[j++] = ch;
 b[j++] = 'D';
 b[j++] = 'L';
 b[j++] = 'E';
 }
 b[j++] = a[i];
 }
 b[j] = '\0';
 strcat(b, "DLEETX");
 printf("stuffed Frame: %s",b);

 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT	EXPECTED
`["ABCDEF",3,"X"]`	`Stuffed Frame: DLESTXABDLEXDLECDEFDLEETX`

ACTUAL OUTPUT

```
stuffed Frame: DLESTXABDLEXDLECDEFDLEETX
```

TEST #2 **PASSED**

INPUT
```
["HELLO",2,"A"]
```

EXPECTED
```
Stuffed Frame: DLESTXHDLEADLEELLODLEETX
```

ACTUAL OUTPUT
```
stuffed Frame: DLESTXHDLEADLEELLODLEETX
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **2/2 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 5:04:26 PM**

Experiment-3

AIM:

The aim of this experiment is to implement the checksum error-detection method used in the Data Link Layer. The program accepts two binary strings of equal length, performs binary addition, applies end-around carry, computes the 1's complement, and displays the checksum.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts two binary strings of equal length from the user.
2. Performs binary addition of the two strings bit by bit from right to left, tracking carry.
3. Prepends any final carry to the sum (end-around carry).
4. Computes the **1's complement** of the full sum.
5. Displays both the sum and the checksum.

INPUT:

Two binary strings of equal length, for example:

```
First string : 10011101
Second string: 11001010
```

OUTPUT:

```
Sum      = 101100111
Checksum = 010011000
```

ALGORITHM:

Step 1

Read two binary strings `a[]` and `b[]` of equal length `n`.

Step 2

Initialize:

```
carry = '0'
i = n - 1
```

Step 3

Loop from `i = n - 1` down to `0`, applying the binary addition truth table:

a[i]	b[i]	Carry In	Sum Bit	Carry Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0

1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Step 4

After the loop:

- If `carry == '1'`,
- Prepend it to the sum.

This produces an `(n + 1)` bit result.

Step 5

Compute the **1's complement**:

- Replace every `0` with `1`.
- Replace every `1` with `0`.

Include the carry bit (if present) while computing the complement.

Step 6

Print the sum and checksum.

EXAMPLES:

EXAMPLE 1 - NO FINAL CARRY:

Input

```
a = 1001
b = 0110
```

Addition

```
1001
0110
----
1111
```

Checksum

```
1's complement of 1111 = 0000
```

Trace (Right to Left)

Position	a	b	Carry In	Sum Bit	Carry Out
3	1	0	0	1	0
2	0	1	0	1	0
1	0	1	0	1	0
0	1	0	0	1	0

Output

```
Sum      = 1111
Checksum = 0000
```

EXAMPLE 2 - WITH FINAL CARRY (END-AROUND CARRY):

Input

```
a = 11001010
b = 10011101
```

Raw Sum

```
11001010
10011101
---------
101100111
```

The final carry is prepended to the sum.

Checksum

```
1's complement of 101100111
=
010011000
```

Output

```
Sum      = 101100111
Checksum = 010011000
```

CONSTRAINTS:

- Both binary strings must be of equal length.
- Strings must contain only:
  - `0`
  - `1`
- Maximum string length is limited by the array size:
  - `20` characters in the original program.
  - `100` characters in the corrected version.
- The program does **not** perform full Internet Checksum carry re-addition.
- It only prepends the final carry to the sum (simplified implementation).

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#include <string.h>
int main()
{
 char a[20], b[20], sum[21],
checksum[21];
 int len, i, carry = 0, temp;
 scanf("%s", a);
 scanf("%s", b);
 len = strlen(a);
 sum[len + 1] = '\0';
 for(i = len - 1; i >= 0; i--)
 {
 temp = (a[i] - '0') + (b[i] - '0') + carry;
 sum[i + 1] = (temp % 2) + '0';
 carry = temp / 2;
 }
 sum[0] = carry + '0';
 for(i = 0; i <= len; i++)
 {
 if(sum[i] == '1')
 checksum[i] = '0';
 else
 checksum[i] = '1';
 }
 checksum[len + 1] = '\0';
printf("Binary addition: A = %s B = %s Sum = %s checksum = %s ", a, b, sum, checksum);
printf("verification (receiver adds data + checksum): If result is all 1s \u2192 1's
complement = all 0s \u2192 no error.");
 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT

```
[1010, 1100]
```

EXPECTED

```
Binary addition: A = 1010 B = 1100 Sum =
10110 Checksum = 01001 Verification
(receiver adds data + checksum): If result
 i s   a l l   1 s  !’   1 ' s   c o m p l e m e n t
error.
```

ACTUAL OUTPUT

```
Binary addition: A = 1010 B = 1100 Sum = 10110 checksum = 01001 verification (receiver adds
 d a t a   +   c h e c k s u m ) :   I f   r e s u l t   i s   a l l   1 s  !’   1 ' s   c o m p l e m
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY — 1/1 Passed

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:24:30 PM**

Experiment-4

AIM:

The aim of this experiment is to implement Hamming Code generation and error correction in C-encoding 4 data bits into a 7-bit codeword, simulating transmission, computing the error syndrome at the receiver, and correcting any single-bit error.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts 4 data bits from the user.
2. Places them at positions 3, 5, 6, and 7 of the 7-bit codeword.
3. Computes parity bits P1, P2, and P4 using even parity (XOR) and places them at positions 1, 2, and 4.
4. Displays the complete 7-bit encoded codeword.
5. Accepts the 7-bit received codeword from the user.
6. Computes syndrome bits C1, C2, and C4 to find the error position.
7. If an error is detected, flips the erroneous bit and displays the corrected codeword.

INPUT:

Data Bits

```
1 0 1 0
```

Received Codeword

```
1 0 1 0 0 1 1
```

(Bit 7 flipped to simulate an error)

OUTPUT:

```
Encoded codeword : 1010010
Error at position: 7
Corrected message: 1010010
```

ALGORITHM:

ENCODING (SENDER SIDE):

Step 1

Accept 4 data bits:

```
D1, D2, D3, D4
```

Step 2

Place data bits at positions:

```
codeword[3] = D1
codeword[5] = D2
codeword[6] = D3
codeword[7] = D4
```

Step 3

Calculate parity bits using XOR (even parity):

```
P1 = codeword[3]  codeword[5]  codeword[7]
```

Place at position 1.

```
P2 = codeword[3]  codeword[6]  codeword[7]
```

Place at position 2.

```
P4 = codeword[5]  codeword[6]  codeword[7]
```

Place at position 4.

Step 4

Display all 7 bits of the codeword.

DECODING (RECEIVER SIDE):

Step 5

Accept 7 received bits.

Step 6

Compute syndrome bits:

```
C1 = rec[1]  rec[3]  rec[5]  rec[7]
```

```
C2 = rec[2]  rec[3]  rec[6]  rec[7]
```

```
C4 = rec[4]  rec[5]  rec[6]  rec[7]
```

Calculate:

```
error_position = (C4  4) + (C2  2) + (C1  1)
```

Step 7

If:

```
error_position == 0
```

Print:

```
No error
```

Step 8

Otherwise:

- Flip the bit at `error_position`.
- Display the corrected codeword.

EXAMPLES:

EXAMPLE 1 - NO ERROR:

Data Bits

```
1 0 1 1
```

Encoding

Position	1	2	3	4	5	6	7
Bit Type	P1	P2	D1	P4	D2	D3	D4
Value	?	?	1	?	0	1	1

Parity Calculation

```
P1 = D1  D2  D4
   = 1  0  1
   = 0
```

```
P2 = D1  D3  D4
   = 1  1  1
   = 1
```

```
P4 = D2  D3  D4
   = 0  1  1
   = 0
```

Codeword

```
0 1 1 0 0 1 1
```

Received

```
0 1 1 0 0 1 1
```

(Same as sent)

Syndrome

```
C1 = 0  1  0  1 = 0
```

```
C2 = 1  1  1  1 = 0
```

```
C4 = 0  0  1  1 = 0
```

Error Position

```
0
```

Result

```
No Error
```

EXAMPLE 2 - SINGLE-BIT ERROR AT POSITION 5:

Sent Codeword

```
0 1 1 0 0 1 1
```

Received Codeword

```
0 1 1 0 1 1 1
```

(Bit 5 flipped)

Syndrome

```
C1 = 0  1  1  1 = 1
```

```
C2 = 1  1  1  1 = 0
```

```
C4 = 0  1  1  1 = 1
```

Error Position

```
(1  4) + (0  2) + (1  1)
= 5
```

Correction

Flip bit at position 5:

```
0 1 1 0 0 1 1
```

Corrected Codeword

```
0 1 1 0 0 1 1
```

EXAMPLE 3 - PROGRAM'S OWN EXAMPLE:

Data Bits

```
1, 0, 1, 0
```

Array Index Mapping Used in Original Code (0-Based)

```
data[0] = 1  (D1 at position 3)
data[1] = 0  (D2)
data[2] = 1  (D3)
data[4] = 0  (D4)
```

Parity Calculation

```
data[6] = data[0] ^ data[2] ^ data[4]
        = 1 ^ 1 ^ 0
        = 0
```

(P1)

```
data[5] = data[0] ^ data[1] ^ data[4]
        = 1 ^ 0 ^ 0
```

```
        = 1
```

(P2)

```
data[3] = data[0] ^ data[1] ^ data[2]
        = 1 ^ 0 ^ 1
        = 0
```

(P4)

Encoded Codeword

```
1 0 1 0 0 1 0
```

Output

```
1010010
```

CONSTRAINTS:

- This implementation handles exactly:
  - 4 data bits
  - 3 parity bits
  - 7-bit Hamming code
- Can detect and correct only **single-bit errors**.
- Can detect but cannot correct **double-bit errors**.
- Extended Hamming Code is required for double-bit error correction.
- Input bits must be integers:

```
0 or 1
```

only.

STUDENT SUBMISSION / CODE:

```
#include<stdio.h>
int main()
{
 int d[4],h[8];
 int r[8];
 int c1,c2,c4,error;
 int i;
 printf("=== Hamming Code - Sender Side ===");
 scanf("%d%d%d%d",&d[0],&d[1],&d[2],&d[3]);
 h[3]=d[0];
 h[5]=d[1];
 h[6]=d[2]; h[7]=d[3];
 h[1]=(h[3]+h[5]+h[7])%2;
```

```
 h[2]=(h[3]+h[6]+h[7])%2;
 h[4]=(h[5]+h[6]+h[7])%2;
 printf(" Coding Table:");
 printf(" Position : 1 2 3 4 5 6 7");
 printf(" Role : P1 P2 D1 P4 D2 D3 D4");
 printf(" Bits :");
 for(i=1;i<=7;i++)
 {
 printf(" %d ",h[i]);
 }
 printf(" Encoded Codeword: ");
 for(i=1;i<=7;i++)
 {
 printf("%d",h[i]);
 }
 printf("=== Hamming Code - Receiver Side ===");
 for(i=1;i<=7;i++)
 {
 scanf(" %d ",&r[i]);
 }
 c1=(r[1]+r[3]+r[5]+r[7])%2;
 c2=(r[2]+r[3]+r[6]+r[7])%2;
 c4=(r[4]+r[5]+r[6]+r[7])%2;
 error=c4*4+c2*2+c1;
 printf(" Syndrome Bits: C4=%d C2=%d C1=%d",c4,c2,c1);
 printf(" Error Position: %d",error);
 printf(" Data Sent :");
 for(i=1;i<=7;i++)
 {
 printf("%d",h[i]);
 }
 printf(" Data Received:");
 for(i=1;i<=7;i++)
 {
 printf("%d", r[i]);
 }
 printf(" Result: Error detected at bit position %d",error);
 if(error!=0)
 r[error]=!r[error];
 printf(" Corrected CodeWord:");
 for(i=1;i<=7;i++)
 {
 printf("%d",r[i]);
 }
 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT

```
1 0 1 1 1 0 1 0 0 1 1
```

EXPECTED

```
=== Hamming Code - Sender Side === Coding Table: Position : 1 2 3 4 5 6 7 Role     : P1 P2 D1 P4 D2 D3 D4 Bits     : 0 1 1 0 0 1 1 Encoded Codeword: 0110011  === Hamming Code - Receiver Side === Syndrome Bits: C4=0 C2=1 C1=1 Error Position: 3 Data Sent     : 0110011 Data Received : 1010011 Result: Error detected at bit position 3 Corrected Codeword: 1000011
```

ACTUAL OUTPUT

```
=== Hamming Code - Sender Side === Coding Table: Position : 1 2 3 4 5 6 7 Role : P1 P2 D1 P4 D2 D3 D4 Bits : 0  1  1  0  0  1  1  Encoded Codeword: 0110011=== Hamming Code - Receiver Side === Syndrome Bits: C4=0 C2=1 C1=1 Error Position: 3 Data Sent :0110011 Data Received:1010011 Result: Error detected at bit position 3 Corrected CodeWord:1000011
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY — 1/1 Passed

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:24:52 PM**

Experiment-5

AIM:

The aim of this experiment is to implement CRC (Cyclic Redundancy Check) for three standard polynomials - **CRC-12, CRC-16, and CRC-CCITT** - on a binary data set, and to demonstrate error detection by flipping a single bit.

PROBLEM STATEMENT:

Write a C program that:

1. Allows the user to choose one of three CRC polynomials:
   - CRC-12 (polynomial `1100000001111`)
   - CRC-16 (polynomial `11000000000000101`)
   - CRC-CCITT (polynomial `10001000000100001`)
2. Accepts a binary string as data (e.g., `"1100110011100011"`).
3. Computes the CRC checksum using binary division (XOR) and appends it to form the codeword.
4. Optionally introduces a single-bit error at a user-specified position.
5. Recalculates the CRC on the possibly corrupted codeword and reports whether an error is detected.

INPUT:

- Choice of polynomial (1, 2, or 3).
- Binary data string (e.g., `1100110011100011`).
- Option to test error detection (0 = yes, 1 = no).
- If error testing, the position of the bit to flip (1-based index).

OUTPUT:

- The generator polynomial used.
- The modified data (data + zeros).
- The computed checksum.
- The final codeword.
- If error is injected, the erroneous codeword and whether the error was detected.

ALGORITHM:

Step-1: Store the chosen generator polynomial as a binary string.
Step-2: Read the input data string and compute its length.
Step-3: Append `deg` zeros to the data (where `deg` = length of generator minus 1).
Step-4: Perform binary division (modulo-2) using a shift-register method:
- Initialise checksum with the first `deg+1` bits of the extended data.
- For each subsequent bit, shift left and XOR with generator if the most significant bit is 1.

Step-5: The final remainder (of length `deg`) is the CRC checksum.
Step-6: Replace the appended zeros with the checksum to form the codeword.
Step-7: For error detection, flip the bit at user-specified position.
Step-8: Re-run CRC division on the codeword (or on the erroneous codeword). If remainder is zero -> no error; else error detected.

EXAMPLES:

Example for CRC-12

- Data: `1100110011100011`
- Polynomial: `1100000001111` (degree 12)
- Extended data (data + 12 zeros): `1100110011100011000000000000`
- Remainder after division: `1101110110001`
- Codeword: `1100110011100011101110110001`

CONSTRAINTS:

- Input data must be a binary string (only 0s and 1s).
- The program uses static character arrays - maximum data length is limited (here 28 characters).

- The CRC division is implemented via a loop that simulates a shift register.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#include <string.h>
void calculateCRC(char data[], char poly[], char remainder[])
{
 char temp[200];
 int dataLen = strlen(data);
 int polyLen = strlen(poly);
 int i, j;
 strcpy(temp, data);
 for (i = 0; i <= dataLen - polyLen; i++)
 {
 if (temp[i] == '1')
 {
 for (j = 0; j < polyLen; j++)
 {
 temp[i + j] = (temp[i + j] == poly[j]) ? '0' : '1';
 }
 }
 }
 for (i = 0; i < polyLen - 1; i++)
 {
 remainder[i] = temp[dataLen - polyLen + 1 + i];
 }
 remainder[polyLen - 1] = '\0';
}
int main()
{
 int choice, errorchoice;
 char data[100];
 char poly[30];
 char modifiedData[200];
 char checksum[30];
 char codeword[200];
 char remainder[30];
 scanf("%d %s %d", &choice, data, &errorchoice);
 printf("1. CRC-12 2. CRC-16 3. CRC-CCITT");
 if (choice == 1)
 {
 strcpy(poly,"1100000001111");
 }
 else if (choice == 2)
 {
 strcpy(poly,"11000000000000101"); }
 else if (choice == 3)
 {
 strcpy(poly,"1000100000100001");
 }
 else
 {
 printf("Invalid Choice");
 return 0;
 }
```

```
 printf(" Generating Polynomial:%s", poly);
 strcpy(modifiedData, data);
 for (int i = 0; i < strlen(poly) - 1; i++)
 strcat(modifiedData, "0");
 printf(" Modified Data: %s", modifiedData);
 calculateCRC(modifiedData, poly, checksum);
 printf(" checksum: %s\n", checksum);
 strcpy(codeword, data);
 strcat(codeword, checksum);
 printf("Final Codeword: %s", codeword);
 printf(" Test error detection? 0 = Yes 1 = No Choice: No Error Detected.");
 if (errorchoice == 0)
 {
 calculateCRC(codeword, poly, remainder);
 int error = 0;
 for (int i = 0; i < strlen(remainder); i++)
 {
 if (remainder[i] == '1')
 {
 error = 1;
 break;
 }
 }
 if (error == 0)
 printf("Choice: No Error Detected.");
 else
 printf("Choice: Error Detected.");
 }
 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT

```
1 100100 1
```

EXPECTED

```
1. CRC-12 2. CRC-16 3. CRC-CCITT Generating
Polynomial: 1100000001111 Modified Data:
1001000000000000000 Checksum: 000101101000
Final Codeword: 100100000101101000  Test
error detection? 0 = Yes 1 = No Choice: No
Error Detected.
```

ACTUAL OUTPUT

```
1. CRC-12 2. CRC-16 3. CRC-CCITT Generating Polynomial:1100000001111 Modified Data:
1001000000000000000 checksum: 000101101000
Final Codeword: 100100000101101000 Test error detection? 0 = Yes 1 = No Choice: No Error
Detected.
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY — 1/1 Passed

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:25:13 PM**

Experiment-6

AIM:

The aim of this experiment is to implement the sender side of the Go-Back-N sliding window protocol in C, demonstrating frame transmission and retransmission when acknowledgements are not received for all frames.

PROBLEM STATEMENT:

Write a C program that simulates the Go-Back-N protocol:

1. Accepts the window size (N) from the user.
2. Transmits frames from 0 to N-1.
3. Accepts the last acknowledgement received from the user.
4. If the acknowledgement number is less than N:
   - Retransmit all frames starting from the acknowledgement number.
5. If the acknowledgement number equals N:
   - Transmission is complete.
6. Continue the process until all frames are acknowledged.

INPUT:

- Window Size (Integer)

```
8
```

- Last Acknowledgement Received (Integer)

```
2
```

```
8
```

OUTPUT:

- Sequence of transmitted frames.
- Retransmitted frames when acknowledgements are incomplete.

Example Output

```
Enter window size: 8

Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
Frame 5 has been transmitted.
Frame 6 has been transmitted.
Frame 7 has been transmitted.

Please enter the last Acknowledgement received: 2
```

```
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
Frame 5 has been transmitted.
Frame 6 has been transmitted.
Frame 7 has been transmitted.

Please enter the last Acknowledgement received: 8

Transmission Completed Successfully.
```

ALGORITHM:

Step-1:
Read the window size `N` from the user.

Step-2:
Initialize the variable:

```
sent = 0
```

where `sent` represents the next frame to be transmitted.

Step-3:
Start a loop.

Step-4:
Transmit all frames in the current window by displaying:

```
Frame i has been transmitted.
```

for each frame from the current position up to `N - 1`.

Step-5:
Ask the user to enter the last acknowledgement received.

Step-6:
Check the acknowledgement value.

- If ACK = N:
  - All frames are acknowledged.
  - Exit the loop.
- If ACK < N:
  - Set:

```
sent = ACK
```

  - Retransmit all frames starting from the unacknowledged frame.

Step-7:
Repeat the process until all frames are acknowledged.

Step-8:
Display a completion message and terminate the program.

EXAMPLES:

Example 1: Complete Acknowledgement

Input

```
Window Size = 4
ACK = 4
```

Output

```
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.

Transmission Completed Successfully.
```

Example 2: Partial Acknowledgement

Input

```
Window Size = 4
ACK = 2
```

Output

```
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.

Please enter the last Acknowledgement received: 2

Frame 2 has been transmitted.
Frame 3 has been transmitted.
```

Example 3: Multiple Retransmissions

Input

```
Window Size = 5

ACK = 1
ACK = 3
ACK = 5
```

Output

```
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.

Retransmitting from Frame 1...

Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.

Retransmitting from Frame 3...

Frame 3 has been transmitted.
Frame 4 has been transmitted.

Transmission Completed Successfully.
```

CONSTRAINTS:

- This is a simulation of the sender side only.
- No actual network communication is performed.
- Acknowledgements are entered manually by the user.
- Timeout handling is not implemented.
- Retransmission is controlled entirely by user-provided acknowledgements.
- Window size should be a positive integer.
- The program continues until all frames are acknowledged.
- Sequence number limits are not enforced in this simulation.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
int main()
{
 int N, ack1, ack2;
 int i;

 scanf("%d %d", &N, &ack1);

 for (i = 0; i < N; i++)
 {
 printf("Frame %d has been transmitted.\n", i);
 }

 if (ack1 == N)
 {
 printf("All frames transmitted successfully.\n");
 return 0;
 } for (i = ack1; i < N; i++)
 {
```

```
 printf("Frame %d has been transmitted.\n", i);
 }
 if (scanf("%d", &ack2) == 1)
 {
 if (ack2 == N)
 {


 for (i = N; i < N + 2; i++)
 {
 printf("Frame %d has been transmitted.\n", i);
 }
 printf("All frames transmitted successfully.\n");
 }
 }

 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 **PASSED**

INPUT
```
4 4
```

EXPECTED
```
Frame 0 has been transmitted. Frame 1 has
been transmitted. Frame 2 has been
transmitted. Frame 3 has been transmitted.
All frames transmitted successfully.
```

ACTUAL OUTPUT
```
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
All frames transmitted successfully.
```

TEST #2 **PASSED**

INPUT
```
4 2 4
```

EXPECTED
```
Frame 0 has been transmitted. Frame 1 has
been transmitted. Frame 2 has been
transmitted. Frame 3 has been transmitted.
Frame 2 has been transmitted. Frame 3 has
been transmitted. Frame 4 has been
transmitted. Frame 5 has been transmitted.
All frames transmitted successfully.
```

ACTUAL OUTPUT
```
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 2 has been transmitted.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
```

ACTUAL OUTPUT (CONTINUED)

```
Frame 5 has been transmitted.
All frames transmitted successfully.
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **2/2 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:25:42 PM**

Experiment-7

AIM:

The aim of this experiment is to implement a simplified simulation of the Selective Repeat Sliding Window Protocol in C, demonstrating how frames are transmitted in windows and how the sender waits for acknowledgements after each window (assuming no transmission errors).

PROBLEM STATEMENT:

Write a C program that:

1. Accepts the window size (`w`) from the user.
2. Accepts the total number of frames to be transmitted.
3. Accepts the frame identifiers (integers).
4. Transmits frames in groups of size `w`.
5. Displays an acknowledgement message after each window of frames is sent.
6. Continues until all frames have been transmitted.

The program assumes:

- No frame loss.
- No frame corruption.
- No retransmissions.

INPUT:

Window Size

```
3
```

Number of Frames

```
5
```

Frame Identifiers

```
12 5 89 4 6
```

OUTPUT:

Frames are displayed window by window, followed by an acknowledgement message.

Example Output

```
Enter window size: 3
Enter number of frames to transmit: 5
Enter 5 frames: 12 5 89 4 6

With sliding window protocol the frames will be sent in the following manner (assuming no
corruption of frames)

After sending 3 frames at each stage sender waits for acknowledgement sent by the receiver
```

```
12 5 89

Acknowledgement of above frames sent is received by sender

4 6

Acknowledgement of above frames sent is received by sender
```

ALGORITHM:

Step-1:
Read the window size `w` from the user.

Step-2:
Read the total number of frames `f`.

Step-3:
Store the frame identifiers in an array.

Step-4:
Display a heading explaining the transmission process.

Step-5:
Iterate through the frames one by one.

Step-6:
Print each frame.

Step-7:
After printing every `w` frames:

- Move to a new line.
- Display the acknowledgement message.
- Continue with the next window.

Step-8:
After all frames are transmitted:

- If the final window contains fewer than `w` frames,
  display an acknowledgement message for the remaining frames.

Step-9:
Terminate the program.

EXAMPLES:

Example 1: Perfect Window Fit

Input

```
Window Size = 3
Frames = 6

Frame List:
10 20 30 40 50 60
```

Output

```
10 20 30

Acknowledgement of above frames sent is received by sender

40 50 60

Acknowledgement of above frames sent is received by sender
```

Example 2: Partial Last Window

Input

```
Window Size = 4
Frames = 5

Frame List:
1 2 3 4 5
```

Output

```
1 2 3 4

Acknowledgement of above frames sent is received by sender

5

Acknowledgement of above frames sent is received by sender
```

Example 3: Window Size = 1

Input

```
Window Size = 1
Frames = 3

Frame List:
7 8 9
```

Output

```
7

Acknowledgement of above frames sent is received by sender

8

Acknowledgement of above frames sent is received by sender

9
```

```
Acknowledgement of above frames sent is received by sender
```

CONSTRAINTS:

- The program is only a simulation of the Selective Repeat transmission pattern.
- No actual networking is performed.
- Retransmissions are not implemented.
- Timeouts are not simulated.
- Selective acknowledgements are not implemented.
- Frame loss and corruption are ignored.
- Window size must be a positive integer.
- Frame identifiers must be integers.
- The sender waits for an acknowledgement after every transmitted window.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>

int main()
{
    int w, n;
        int frames[100];
            int i, j, count;

                scanf("%d", &w);
                    scanf("%d", &n);

                        for (i = 0; i < n; i++)
                            {
                                    scanf("%d", &frames[i]);
                                        }

                                            printf("With sliding window protocol the frames
will be sent in the following manner (assuming no corruption of frames)");
                                                printf("After sending %d frames at each
stage sender waits for acknowledgement sent by the receiver", w);

                                                    for (i = 0; i < n; i = i + w)
                                                        {
                                                                count = 0;
                                                                        for (j = i; j < n &&
j < i + w; j++)
                                                                                {

if (count > 0)

           {

                           printf(" ");

                                       }

                                                   printf("%d", frames[j]);
```

```
                                                        count++;

                                                                }

                                                                        if (count <
w)

                                                                                {


      printf(" ");

              }

                      printf("Acknowledgement of above frames sent is received by sender");

                          }


                              return 0;

                              }
```

OUTPUT & ANALYSIS:

TEST #1 **PASSED**

INPUT

```
[3,6,[1,2,3,4,5,6]]
```

EXPECTED

```
With sliding window protocol the frames will be sent in the following manner (assuming no corruption of frames)After sending 3 frames at each stage sender waits for acknowledgement sent by the receiver1 2 3Acknowledgement of above frames sent is received by sender4 5 6Acknowledgement of above frames sent is received by sender
```

ACTUAL OUTPUT

```
With sliding window protocol the frames will be sent in the following manner (assuming no corruption of frames)After sending 3 frames at each stage sender waits for acknowledgement sent by the receiver1 2 3Acknowledgement of above frames sent is received by sender4 5 6Acknowledgement of above frames sent is received by sender
```

TEST #2 **PASSED**

INPUT

```
[4, 7, [10, 20, 30, 40, 50, 60, 70]]
```

EXPECTED

```
With sliding window protocol the frames will be sent in the following manner (assuming no corruption of frames)After sending 4 frames at each stage sender waits for acknowledgement sent by the receiver10 20 30 40Acknowledgement of above frames sent is received by sender50 60 70 Acknowledgement of above frames sent is received by sender
```

ACTUAL OUTPUT

```
With sliding window protocol the frames will be sent in the following manner (assuming no corruption of frames)After sending 4 frames at each stage sender waits for acknowledgement sent by the receiver10 20 30 40Acknowledgement of above frames sent is received by sender50 60 70 Acknowledgement of above frames sent is received by sender
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **2/2 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:26:01 PM**

Experiment-8

AIM:

The aim of this experiment is to implement the Stop-and-Wait ARQ Protocol in C using standard GCC-compatible libraries, simulating:

- Frame transmission and acknowledgement
- Timeout and retransmission mechanisms
- Error handling (corrupted frames)
- Alternating sequence numbers (0 and 1)

PROBLEM STATEMENT:

Write a C program that simulates the Stop-and-Wait ARQ Protocol between a sender and a receiver. The program should:

- Transmit a sequence of frames (up to `TOT_PACKETS = 8`).
- Use alternating sequence numbers (0 and 1).
- Simulate random transmission errors using a random number generator.
- Implement a timeout mechanism to detect lost frames or acknowledgements.
- Retransmit frames when acknowledgements are not received.
- Deliver correctly received packets to the receiver.
- Display messages for frame transmission, acknowledgements, timeouts, retransmissions, and errors.
- Terminate after all packets are successfully transmitted.

INPUT:

No user input is required.

The simulation runs automatically and generates random transmission errors using:

```
rand() % 4
```

OUTPUT:

Console messages showing:

- Sender transmitting frames
- Receiver receiving packets
- Acknowledgements being sent
- Corrupted frame detection
- Timeout events
- Frame retransmissions
- Successful disconnection after all packets are delivered

Example Output

```
Stop-and-Wait ARQ Simulation

SENDER   : Info = 1, Seq = 0
RECEIVER : Packet 1 received, ACK sent

SENDER   : Info = 2, Seq = 1
RECEIVER : Garbled Frame
SENDER   : Timeout
SENDER   : Retransmitting

SENDER   : Info = 2, Seq = 1
```

```
RECEIVER : Packet 2 received, ACK sent

...

DISCONNECTED
```

ALGORITHM / PROCEDURE:

Step-1

Include required header files:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

Step-2

Define constants:

```
#define TIMEOUT 5
#define MAX_SEQ 1
#define TOT_PACKETS 8
```

Step-3

Define data structures:

- `packet` structure to store packet data
- `frame` structure containing:
  - sequence number
  - acknowledgement number
  - packet information
  - error status

Step-4

Declare global variables:

```
DATA
turn
DISCONNECT
```

Step-5

Implement `from_network_layer()`:

- Generate packet data.
- Increment packet number.

Step-6

Implement `to_network_layer()`:

- Deliver received packets.
- Check for successful completion of transmission.

Step-7

Implement `to_physical_layer()`:

- Simulate transmission errors using:

```
rand() % 4
```

- Store the transmitted frame.

Step-8

Implement `from_physical_layer()`:

- Read the transmitted frame from the physical layer.

Step-9

Implement timeout handling:

- Detect missing acknowledgements.
- Trigger retransmission of frames.

Step-10

Implement sender logic:

- Create and transmit frames.
- Assign alternating sequence numbers.
- Retransmit frames after timeout.

Step-11

Implement receiver logic:

- Receive frames.
- Detect corrupted frames.
- Send acknowledgements for valid frames.

Step-12

Initialize random number generator:

```
srand(time(NULL));
```

Step-13

Execute the simulation until all packets are successfully transmitted and acknowledged.

EXAMPLES:

EXAMPLE OF ALTERNATING SEQUENCE NUMBERS:

1. Packet 1 transmitted with sequence number `0`
2. ACK received
3. Packet 2 transmitted with sequence number `1`
4. ACK received
5. Packet 3 transmitted with sequence number `0`

EXAMPLE OF RETRANSMISSION:

1. Sender transmits frame with sequence number `1`
2. Frame becomes corrupted
3. Receiver reports error
4. Sender times out
5. Sender retransmits the same frame
6. Receiver successfully receives the frame and sends ACK

CONSTRAINTS:

- Output varies for each execution because transmission errors are generated randomly.
- Uses pseudo-random error generation via `rand()`.
- Simulates communication through a shared frame structure.
- Implements one-way data transfer from sender to receiver.
- Designed for educational simulation of Stop-and-Wait ARQ.
- Compatible with GCC, Clang, Piston, and other modern C compilers.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
int main() {
int n, i;
printf("Enter number of frames: ");
scanf("%d", &n);
 for (i = 1; i <= n; i++) {
 printf("\nSender: Sending Frame %d...\n", i);
 if (i == 3) {
 printf("Frame %d transmission failed!\n", i);
 printf("Sender: Timeout occurred.\n");
 printf("Sender: Retransmitting Frame %d...\n", i);
 }
 printf("Receiver: Frame %d received successfully.\n", i);
 printf("Receiver: Sending ACK %d.\n", i);
 printf("Sender: ACK %d received.\n", i);
 }
 printf("\nAll frames transmitted successfully.\n");
 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT	EXPECTED
-	Successful Execution

ACTUAL OUTPUT

```
All frames transmitted successfully.
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY — 1/1 Passed

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:26:25 PM**

Experiment-9

AIM:

The aim of this experiment is to implement the **Leaky Bucket Algorithm** for congestion control in C, simulating packet arrivals, queuing, constant output rate transmission, and packet drops due to buffer overflow.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts **bucket size** (maximum queue length) and **output rate** (packets transmitted per second) from the user.
2. For each time interval (second), accepts the number of incoming packets (or stops when the user chooses to quit).
3. Simulates the Leaky Bucket behaviour:
   - Adds incoming packets to the queue.
   - Sends out up to `output rate` packets per second.
   - If the queue exceeds bucket size, excess packets are dropped.
4. Displays a table showing:
   - Second number
   - Packets received
   - Packets sent
   - Packets dropped
   - Packets remaining in queue (after transmission and drops)
5. Continues until all queued packets have been transmitted (even after the user stops entering new arrivals).

INPUT:

- Bucket size (integer)
- Output rate (integer)
- For each second:
  - Number of incoming packets (integer)
- User choice to continue or stop entering new arrivals

OUTPUT:

- A formatted table showing the state of the bucket at each second.

Example Output

```
Enter bucket size: 5
Enter output rate: 3

Enter number of packets coming at second 1: 1
Enter 1 to continue or 0 to quit: 1

Enter number of packets coming at second 2: 2
Enter 1 to continue or 0 to quit: 0

Second  Received  Sent  Dropped  Remaining
1       1         1     0        0
2       2         2     0        0
```

ALGORITHM / PROCEDURE:

Step-1

Include the required header files:

```
#include <stdio.h>
#include <stdlib.h>
```

Step-2

Define the `MIN(x,y)` macro to find the smaller of two numbers.

Step-3

Declare the following variables:

- `orate` - Output rate (packets per second)
- `drop` - Number of packets dropped
- `cap` - Bucket capacity
- `x` - Temporary variable for calculations
- `count` - Current number of packets in the bucket
- `inp[10]` - Array storing incoming packet counts per second
- `i` - Index counter
- `nsec` - Total number of seconds user entered packets for
- `ch` - User choice to continue or quit

Step-4

Accept bucket size (`cap`) and output rate (`orate`) from the user.

Step-5

Use a `do-while` loop to accept incoming packets for each second until the user chooses to quit.

Step-6

Set:

```
nsec = i;
```

to represent the total number of seconds with incoming packets.

Step-7

Print the table headers.

Step-8

Loop through each second (including extra seconds required to drain the queue):

- Print the second number.
- Display packets received (`inp[i]`).
- Calculate packets sent:

```
MIN((inp[i] + count), orate)
```

- Calculate the new queue size:

```
x = inp[i] + count - orate;
```

- If:

```
x > 0
```

- If:

```
x > cap
```

  - Overflow occurs:

```
count = cap;
drop = x - cap;
```

- Otherwise:

```
count = x;
drop = 0;
```

- Else:

```
count = 0;
drop = 0;
```

- Display packets dropped and remaining packet count.

Step-9

End the program.

EXAMPLES:

EXAMPLE 1: NORMAL OPERATION (NO OVERFLOW):

- Bucket size = 5
- Output rate = 3

Second 1

- 1 packet arrives
- 1 packet sent
- 0 packets remain

Second 2

- 2 packets arrive
- 2 packets sent
- 0 packets remain

EXAMPLE 2: BURSTY TRAFFIC CAUSING OVERFLOW:

- Bucket size = 5
- Output rate = 2

Second 1

- 5 packets arrive
- Sends 2 packets
- 3 packets remain in queue

Second 2

- 5 more packets arrive
- Queue becomes 8 packets
- Bucket can hold only 5 packets
- 3 packets are dropped

Transmission

```
Sent = min(5, 2) = 2
Remaining = 3
```

EXAMPLE 3: DRAINING THE QUEUE:

After the user stops entering packets, the loop continues until:

```
count == 0
```

sending `orate` packets each second until the queue becomes empty.

CONSTRAINTS:

- The program uses a fixed-size array for incoming packets:

```
inp[10]
```

For more seconds, increase the array size or use dynamic allocation.

- The `MIN` macro is used for simplicity and can be replaced with a function.
- The program assumes integer packets (no fractional packets).
- The simulation continues until the queue is empty, even after user input stops.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MIN(x, y) (x > y ? y : x)
int main()
{
int orate, cap, x, count = 0, i = 0, nsec, ch;
int inp[100] = {0};
printf("\n Enter bucket size : ");
scanf("%d", &cap);
printf("\n Enter output rate : ");
scanf("%d", &orate);
do
{
if(i >= 100)
{
printf("Maximum number of input seconds reached. Exiting input loop.\n"); break;
```

```
}
printf("Enter number of packets coming at second %d : ", i + 1);
scanf("%d", &inp[i]);
i++;
printf("Enter 1 to continue or 0 to quit: ");
scanf("%d", &ch);
} while(ch == 1);
nsec = i;
printf("Second \t Received \t Sent \t Dropped \t Remaining");
for(i = 0; count > 0 || i < nsec; i++)
{
int current_received = (i < nsec) ? inp[i] : 0;
int sent = MIN(current_received + count, orate);
x = current_received + count - sent;
int dropped = 0;
if(x > cap)
{
dropped = x - cap;
count = cap;
}
else
{
count = x;
}
printf("%d\t %d\t %d\t %d\t %d", i + 1, current_received, sent, dropped, count);
}
return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 **PASSED**

INPUT

```
[10,4,8,1,6,1,12,0]
```

EXPECTED

```
Second          Received          Sent          Dropped
Remaining1          8                   4
                  63          12                  4
65          0                 4                   0
```

ACTUAL OUTPUT

```
Second          Received          Sent          Dropped          Remaining1          8          4
4          0          65          0          4          0          26          0          2          0
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **1/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:26:51 PM**

Experiment-10

AIM:

The aim of this experiment is to implement **Dijkstra's Algorithm** in C to compute the shortest paths from a given source vertex to all other vertices in a weighted, non-negative graph using an adjacency matrix representation.

PROBLEM STATEMENT:

Write a C program that:

1. Represents a **weighted graph** with 9 vertices using an adjacency matrix.
2. Implements Dijkstra's Algorithm to find the shortest path from the **source vertex 0** to all other vertices.
3. Uses an array `dist[]` to store distances and a boolean array `sptSet[]` to track processed vertices.
4. Defines a helper function `minDistance()` to find the vertex with the minimum distance among unprocessed vertices.
5. Outputs the shortest distance from the source to each vertex.

INPUT:

- No user input - the graph is hardcoded in the program.

OUTPUT:

- A table showing the **distance from the source vertex** (vertex 0) to each vertex (0 to 8).

Example Output

```
Vertex   Distance from Source

0        0
1        6
2        14
3        21
4        21
5        11
6        9
7        8
8        15
```

ALGORITHM / PROCEDURE:

Step-1

Include the necessary header files:

```
#include <stdio.h>
#include <limits.h>
```

(`limits.h` is used for `INT_MAX`)

Step-2

Define the number of vertices:

```
#define V 9
```

Step-3

Implement `minDistance()`:

- Finds the unprocessed vertex having the smallest distance value.
- Returns the index of that vertex.

Step-4

Implement `printSolution()`:

- Prints the contents of the `dist[]` array.
- Displays the shortest distance from the source vertex to every other vertex.

Step-5

Implement `dijkstra()`:

Initialize Arrays

- Set all elements of `dist[]` to `INT_MAX` (infinity).
- Set all elements of `sptSet[]` to `false`.

Set Source Distance

```
dist[src] = 0;
```

Main Loop

Repeat `V - 1` times:

1. Call `minDistance()` to select the vertex `u` with the smallest distance value.
2. Mark `u` as processed:

```
sptSet[u] = true;
```

3. For every neighbour `v` of `u`:
   - If `v` is not processed.
   - An edge exists between `u` and `v`.
   - `dist[u]` is not `INT_MAX`.

   Then check:

```
if (dist[u] + graph[u][v] < dist[v])
```

If true:

```
dist[v] = dist[u] + graph[u][v];
```

Display Result

Call:

```
printSolution(dist);
```

to print the shortest path distances.

Step-6

In `main()`:

- Define the graph using an adjacency matrix.
- Call:

```
dijkstra(graph, 0);
```

where `0` is the source vertex.

EXAMPLES:

EXAMPLE ON A SIMPLE GRAPH (3 VERTICES):

Graph

```
0  1 : 4
0  2 : 2
1  2 : 1
```

Output

```
Vertex   Distance from Source

0        0
1        4
2        2
```

CONSTRAINTS:

- All edge weights must be **non-negative**.
- The graph uses an adjacency matrix and therefore requires **O(V²)** space.
- The algorithm assumes the graph is connected. If not, disconnected vertices retain distance `INT_MAX`.
- The source vertex is fixed to `0` in this program (can be modified to accept user input).

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#include <limits.h>
#define V 9
// Find the vertex with minimum distance
int minDistance(int dist[], int visited[])
{
 int min = INT_MAX;
 int min_index = -1;
 for (int v = 0; v < V; v++)
 {
 if (!visited[v] && dist[v] <= min)
 { min = dist[v];
 min_index = v;
 }
 }
```

```
 return min_index;
}
// Dijkstra's Algorithm
void dijkstra(int graph[V][V], int source)
{
 int dist[V];
 int visited[V];
 // Initialize distances and visited array
 for (int i = 0; i < V; i++)
 {
 dist[i] = INT_MAX;
 visited[i] = 0;
 }
 dist[source] = 0;
 // Find shortest paths
 for (int count = 0; count < V - 1; count++)
 {
 int u = minDistance(dist, visited);
 visited[u] = 1;
 for (int v = 0; v < V; v++)
 {
 if (!visited[v] &&
 graph[u][v] &&
 dist[u] != INT_MAX &&
 dist[u] + graph[u][v] < dist[v])
 {
 dist[v] = dist[u] + graph[u][v];
 }
 }
 }
 // Display result
 printf("Vertex\tDistance from Source\n");
 for (int i = 0; i < V; i++)
 {
 printf("%d\t%d\n", i, dist[i]);
 }
}
int main()
{
 int graph[V][V] = {
 // 0 1 2 3 4 5 6 7 8
 { 0, 6, 0, 0, 0, 0, 0, 8, 0 }, // 0 { 6, 0, 8, 0, 0, 0, 0, 0, 0 }, // 1
 { 0, 8, 0, 7, 0, 0, 0, 0, 0 }, // 2
 { 0, 0, 7, 0, 0, 0, 0, 0, 0 }, // 3
 { 0, 0, 0, 0, 0, 10, 0, 0, 0 }, // 4
 { 0, 0, 0, 0, 10, 0, 2, 0, 0 }, // 5
 { 0, 0, 0, 0, 0, 2, 0, 1, 0 }, // 6
 { 8, 0, 0, 0, 0, 0, 1, 0, 7 }, // 7
 { 0, 0, 0, 0, 0, 0, 0, 7, 0 } // 8
 };
 // Source vertex = 0
 dijkstra(graph, 0);
 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 — **PASSED**

INPUT

```
-
```

EXPECTED

```
Successful Execution
```

ACTUAL OUTPUT

```
Vertex        Distance from Source
0        0
1        6
2        2147483647
3        13
4        2147483647
5        23
6        2147483647
7        8
8        2147483647
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY — **1/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:27:14 PM**

Experiment-11

AIM:

The aim of this experiment is to implement the **Distance Vector Routing Algorithm** based on the Bellman-Ford equation to compute routing tables for all nodes in a network, where edge weights represent link delays.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts the number of nodes in the network.
2. Accepts a **cost matrix** representing link delays.
3. Initializes routing tables with direct link costs.
4. Iteratively applies the Bellman-Ford update rule.
5. Continues until no further updates occur (convergence).
6. Prints final routing tables for each node showing:
   - Destination node
   - Next-hop (via)
   - Total cost

INPUT:

- Number of nodes
- Cost matrix (`N × N`)
  - `cost[i][j]` = link cost from node `i` to `j`
  - Use a large value (e.g., `999`) for no direct link

OUTPUT:

For each router:

- Destination node
- Next hop (via)
- Distance (cost)

Example Output

```
For router 1

node 1 via 1 Distance 0
node 2 via 2 Distance 2
node 3 via 2 Distance 3

For router 2

node 1 via 1 Distance 2
node 2 via 2 Distance 0
node 3 via 3 Distance 1

For router 3

node 1 via 2 Distance 3
node 2 via 2 Distance 1
node 3 via 3 Distance 0
```

ALGORITHM / PROCEDURE:

Step-1

Define structure:

```
struct node {
    dist[20];   // distance vector
    from[20];   // next-hop vector
};
```

Step-2

Read input:

- Number of nodes
- Cost matrix

Step-3

Initialization:

```
rt[i].dist[j] = costmat[i][j]
rt[i].from[j] = j
```

Step-4

Iterative update loop:

- Set `count = 0`
- For each node `i`
- For each destination `j`
- For each intermediate node `k`

If:

```
rt[i].dist[j] > costmat[i][k] + rt[k].dist[j]
```

Then:

```
rt[i].dist[j] = costmat[i][k] + rt[k].dist[j]
rt[i].from[j] = k
count++
```

Step-5

Repeat until:

```
count == 0
```

(no further updates -> convergence)

Step-6

Print final routing tables.

EXAMPLES:

3-NODE EXAMPLE:

Cost Matrix

```
0 2 7
2 0 1
7 1 0
```

Result Insight

- Best path 1 -> 3:
  - Direct = 7
  - Via 2 = 2 + 1 = 3 ' (selected)
- Best path 3 -> 1:
  - Via 2 = 1 + 2 = 3 '

CONSTRAINTS:

- Network must be connected for meaningful convergence.
- Use a large value (e.g., 999) for no direct link.
- Works for both directed and undirected graphs.
- Node indexing may start from 0 internally but is displayed as 1-based for readability.

STUDENT SUBMISSION / CODE:

```
#include <stdio.h>
#define INF 999
int main() {
 int n;
 if (scanf("%d", &n) != 1) return 0;
 int cost[20][20], dist[20][20], next[20][20];
 // Read the cost matrix
 for (int i = 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 scanf("%d", &cost[i][j]);
 dist[i][j] = cost[i][j];
 if (i == j) {
 next[i][j] = i;
 } else if (cost[i][j] != INF) {
 next[i][j] = j;
 } else {
 next[i][j] = -1;
 }
 }
 }
 // Bellman-Ford Routing Algorithm
 int updated;
 do { updated = 0;
 for (int i = 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 for (int k = 0; k < n; k++) {
 if (cost[i][k] != INF && dist[k][j] != INF) {
 if (dist[i][j] > cost[i][k] + dist[k][j]) {
```

```
 dist[i][j] = cost[i][k] + dist[k][j];
 next[i][j] = k;
 updated = 1;
 }
 }
 }
 }
 }
 } while (updated);
 // Print the output exactly as expected by the test case
 for (int i = 0; i < n; i++) {
 printf("For router %d ", i + 1);
 for (int j = 0; j < n; j++) {
 printf("node %d via %d Distance %d ", j + 1, next[i][j] + 1, dist[i][j]);
 }
 }
 printf("\n");
 return 0;
}
```

OUTPUT & ANALYSIS:

TEST #1 **PASSED**

INPUT

```
4 0 2 7 999 2 0 1 3 7 1 0 2 999 3 2 0
```

EXPECTED

```
For router 1        node 1 via 1 Distance 0         nod
via 2 Distance 2         node 3 via 2 Distance 3
        node 4 via 2 Distance 5 For router 2        nod
via 1 Distance 2         node 2 via 2 Distance 0
        node 3 via 3 Distance 1         node 4 via 4
Distance 3 For router 3        node 1 via 2
Distance 3         node 2 via 2 Distance 1        node
via 3 Distance 0         node 4 via 4 Distance 2
For router 4        node 1 via 2 Distance 5         nod
via 2 Distance 3         node 3 via 3 Distance 2
        node 4 via 4 Distance 0
```

ACTUAL OUTPUT

```
For router 1 node 1 via 1 Distance 0 node 2 via 2 Distance 2 node 3 via 2 Distance 3 node 4
via 2 Distance 5 For router 2 node 1 via 1 Distance 2 node 2 via 2 Distance 0 node 3 via 3
Distance 1 node 4 via 4 Distance 3 For router 3 node 1 via 2 Distance 3 node 2 via 2
Distance 1 node 3 via 3 Distance 0 node 4 via 4 Distance 2 For router 4 node 1 via 2
Distance 5 node 2 via 2 Distance 3 node 3 via 3 Distance 2 node 4 via 4 Distance 0
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **1/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:50:09 PM**

Experiment-12

AIM:

The aim of this experiment is to implement a **Broadcast Tree** for a subnet of hosts using **Prim's Minimum Spanning Tree (MST) Algorithm**, where edge weights represent delays or communication costs between hosts.

PROBLEM STATEMENT:

Write a C program that:

1. Accepts the **number of nodes** (hosts) in the subnet.
2. Accepts a **cost matrix** (adjacency matrix) representing link costs between nodes.
3. Computes the **Minimum Spanning Tree (MST)** using Prim's Algorithm.
4. Outputs:
   - The total **minimum cost** of the broadcast tree.
   - The edges included in the tree along with their weights.
5. Uses a parent array to track the nodes included in the tree.

INPUT:

- Number of nodes (`n`)
- Cost matrix (`n × n`), where:

```
edge[i][j]
```

represents the cost between node `i` and node `j`.

Use:

```
99
```

(or another large value) to represent no direct connection.

OUTPUT:

- Minimum cost of the broadcast tree.
- List of edges included in the Minimum Spanning Tree.

Example Output

```
Minimum cost is 13

Minimum spanning tree is

A C 1
C B 2
C D 3
```

ALGORITHM / PROCEDURE:

Step-1

Accept:

- Number of nodes (`n`)
- Cost matrix (`edge[][]`)

from the user.

Step-2

Initialize the parent array:

```
parent[i] = -1;
```

This indicates that initially all nodes belong to separate components.

Step-3

Repeat until `n - 1` edges are selected:

Find Minimum Edge

- Search the entire cost matrix.
- Find the edge with the minimum weight connecting two nodes.

Let:

```
u, v
```

be the endpoints of the minimum-weight edge.

Check Components

If:

```
find(u) != find(v)
```

the vertices belong to different components.

Then:

- Add the edge to the MST.
- Store:

```
t[i][0] = u;
t[i][1] = v;
```

- Add edge cost to:

```
mincost += edge[u][v];
```

- Merge the components using union operation:

```
parent[p] = q;
```

Otherwise

If both vertices belong to the same component:

```
t[i][0] = -1;
t[i][1] = -1;
```

The edge is discarded because it would create a cycle.

Reset Minimum

After each iteration:

```
min = 99;
```

to search for the next smallest edge.

Step-4

Print:

- Total minimum cost (`mincost`)
- All valid edges stored in the MST

EXAMPLES:

EXAMPLE 4-NODE GRAPH:

Cost Matrix

```
A   B   C   D

A      0   2   1   4
B      2   0   3   5
C      1   3   0   3
D      4   5   3   0
```

(99 may be used instead of missing edges.)

Minimum Spanning Tree

Selected edges:

```
A - C (1)
A - B (2)
C - D (3)
```

Total Cost

```
1 + 2 + 3 = 6
```

Broadcast Tree

```
A
```

```
 / \
C   B
|
D
```

CONSTRAINTS:

- The graph must be **connected** (all nodes reachable).
- Use:

```
99
```

as infinity for no direct link.

- Node indices start from `0` internally but are displayed as:

```
A, B, C, D, ...
```

for readability.

- The parent array implements a simple Union-Find structure without path compression.
- The Minimum Spanning Tree contains exactly:

```
n - 1
```

edges for a graph with `n` nodes.

STUDENT SUBMISSION / CODE:

```
#include<stdio.h>

int parent[50];

int find(int l){
    while(parent[l]>=0) l=parent[l];
        return l;
        }
        void sunion(int l,int m){
            parent[l]=m;
            }

            int main(){
                int n, edge[50][50];
                    int i,j;
                        scanf("%d",&n);
                            for(i=0;i<n;i++)
                                    for(j=0;j<n;j++)
                                                scanf("%d",&edge[i][j]);

                                                    printf("Node labels: ");
```

```
                                                    for(i=0;i<n;i++) printf("%c ",65+i);
                                                        printf(" ");
                                                            for(i=0;i<n;i++) printf("Row
%c: ",65+i);

                                                                for(i=0;i<n;i++)
parent[i]=-1;

                                                                    // Collect edges
                                                                        int ek1[100],
ek2[100], ew[100], ecnt=0;

for(i=0;i<n;i++){

for(j=i+1;j<n;j++){

       if(edge[i][j]!=0 && edge[i][j]!=99){

                        ek1[ecnt]=i; ek2[ecnt]=j; ew[ecnt]=edge[i][j]; ecnt++;

                                    }

                                            }

                                                }

                                                    // sort by weight

                                                        for(i=0;i<ecnt-1;i++)

                                                                for(j=0;j<ecnt-1-i;j++)


if(ew[j]>ew[j+1]){


int t=ew[j]; ew[j]=ew[j+1]; ew[j+1]=t;


               t=ek1[j]; ek1[j]=ek1[j+1]; ek1[j+1]=t;


                                t=ek2[j]; ek2[j]=ek2[j+1]; ek2[j+1]=t;


                                            }


                                                int mincost=0;


                                                    int res_u[50], res_v[50], rcnt=0;


                                                        for(i=0;i<ecnt;i++){
```

```
                                                        int p=find(ek1[i]),
q=find(ek2[i]);

                                                                if(p!=q){


sunion(p,q);

 mincost+=ew[i];

             res_u[rcnt]=ek1[i]; res_v[rcnt]=ek2[i]; rcnt++;

                         if(rcnt==n-1) break;

                                 }

                                     }


                                         printf("Minimum Cost = %d ",mincost);

                                             printf("Minimum Spanning Tree (Broadcast Tree):
");

                                                 for(i=0;i<rcnt;i++){

                                                         printf("%c -- %c
",65+res_u[i],65+res_v[i]);

                                                             }
```

```
    return 0;


}
```

OUTPUT & ANALYSIS:

TEST #1 **PASSED**

INPUT

```
4  0 10 6 5 10 0 99 15 6 99 0 4 5 15 4 0
```

EXPECTED

```
Node labels: A B C D  Row A: Row B: Row C:
Row D:  Minimum Cost = 19  Minimum Spanning
Tree (Broadcast Tree): C -- D A -- D A -- B
```

ACTUAL OUTPUT

```
Node labels: A B C D  Row A: Row B: Row C: Row D: Minimum Cost = 19 Minimum Spanning Tree
(Broadcast Tree): C -- D A -- D A -- B
```

Performance Summary

Solution evaluation complete

100	100%	0ms
POINTS	RATE	TIME

ACCURACY **1/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 10:12:29 PM**

Experiment-13

AIM:

The aim of this experiment is to study **Wireshark** - a network protocol analyzer used for packet capture, analysis, and network troubleshooting. The study includes installation, packet capture, packet analysis, filtering, and use of statistics tools.

PROBLEM STATEMENT:

1. Install Wireshark on a Windows or Linux system.
2. Capture live network traffic using Wireshark (GUI and command line).
3. Apply capture and display filters to isolate specific traffic.
4. Analyze captured packets and identify protocols, source/destination addresses, and payload content.
5. Use Wireshark's statistics and analysis tools to understand network traffic patterns.
6. Save and open capture files for offline analysis.

INPUT:

- Live network interface with internet connectivity
- (Optional) Pre-saved capture file (.pcap or .pcapng)

OUTPUT:

- Captured packets displayed in the Wireshark GUI
- Filtered traffic based on user-defined criteria
- Statistics reports (Protocol Hierarchy, Endpoints, Conversations)
- Saved capture file for offline analysis

Example Output (Packet List):

```
No.   Time       Source            Destination       Protocol   Length   Info
1     0.000000   192.168.1.10      192.168.1.1       TCP        66       50152  443 [SYN]
Seq=0
2     0.012345   192.168.1.1       192.168.1.10      TCP        66       443  50152 [SYN,
ACK] Seq=0 Ack=1
3     0.012456   192.168.1.10      192.168.1.1       TCP        54       50152  443 [ACK]
Seq=1 Ack=1
4     0.023456   192.168.1.10      192.168.1.1       HTTP       456      GET /index.html
HTTP/1.1
```

ALGORITHM / PROCEDURE:

Part A: Installation of Wireshark

Step-1: Open a web browser and navigate to the official Wireshark website: https://www.wireshark.org/
Step-2: Click on the **Download** button and select the Windows installer (64-bit or 32-bit as per system).
Step-3: Save the installer file and run it.
Step-4: Follow the installation wizard:
- Accept the license agreement.
- Select the installation directory (default is fine).
- Select components: Wireshark, TShark, plugins, and tools.
- **Important**: Install **Npcap** or **WinPcap** when prompted (required for packet capture).
- Complete the installation.

Step-5: Launch Wireshark from the Start Menu or desktop shortcut.

For Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install wireshark
```

During installation, you may be prompted to allow non-root users to capture packets. Select "Yes" if you want to allow this.

Part B: Packet Capture Using Wireshark (GUI)

Step-1: Launch Wireshark.
Step-2: In the main window, a list of available network interfaces is displayed. The interface with traffic activity will show a graph of packets.
Step-3: Click on the interface name (e.g., Ethernet, Wi-Fi) or select **Capture -> Interfaces** to see all interfaces.
Step-4: Click the **Start** button (blue shark fin) to begin capturing packets.
Step-5: The capture dialog will show the number of packets increasing as traffic flows.
Step-6: Once sufficient packets have been captured (or after reproducing the problem to be analyzed), click the **Stop** button (red square).
Step-7: The captured packets will appear in the Packet List pane.
Step-8: To save the capture, go to **File -> Save As** and choose a location and filename (default format `.pcapng`).

Part C: Starting Wireshark from Command Line

Step-1: Open Command Prompt (Windows) or Terminal (Linux).
Step-2: View help and command-line options:

```
wireshark -h
```

Step-3: Capture packets from a specific interface with a capture filter:

```
wireshark -i <interface> -f "host 192.168.1.10" -w capture.pcap
```

Step-4: Open a saved capture file:

```
wireshark -r capture.pcap
```

Step-5: Apply a display filter when opening:

```
wireshark -r capture.pcap -Y "http"
```

Part D: Viewing Captured Traffic

Step-1: In the Packet List pane, click on any packet to select it.
Step-2: The Packet Details pane will expand, showing the protocol layers of the selected packet (e.g., Frame, Ethernet, IP, TCP, HTTP).
Step-3: Click on the `+` signs to expand each layer and view detailed fields (source/destination addresses, ports, flags, etc.).
Step-4: The Packet Bytes pane displays the raw hexadecimal and ASCII representation of the packet.
Step-5: Use the scroll wheel or scrollbar to navigate through the packet list.

Part E: Applying Filters

Step-1: In the **Display Filter** bar (above the Packet List pane), type a filter expression.

Example: To show only HTTP traffic:

```
http
```

Example: To show traffic from a specific IP:

```
ip.src == 192.168.1.10
```

Step-2: Press **Enter** or click the **Apply** arrow. The Packet List will update to show only matching packets.
Step-3: To clear the filter, click the **X** button next to the filter bar.

Capture Filters:

- Go to **Capture -> Options**.
- In the **Capture Filter** field, enter a BPF filter (e.g., `host 192.168.1.10`).
- Click **Start** to begin capture with the filter applied.

Part F: Analysis and Statistics

Step-1: Go to the **Statistics** menu to access various analysis tools.

Tool	Description
Protocol Hierarchy	Displays a breakdown of protocols in the capture (percentage of packets/bytes per protocol).
Endpoints	Shows all endpoints (MAC, IP, ports) and their traffic statistics.
Conversations	Shows traffic between pairs of endpoints.
IO Graphs	Plots traffic over time (packets/sec, bytes/sec).
Flow Graph	Visualizes the sequence of packets between hosts.
TCP Stream Graph	Analyzes TCP streams (timing, throughput, etc.).
Packet Lengths	Displays the distribution of packet lengths.

Step-2: Click on **Statistics -> Protocol Hierarchy** to view the breakdown.
Step-3: Click on **Statistics -> Endpoints** to see all communicating endpoints.
Step-4: Click on **Statistics -> Conversations** to see traffic between host pairs.

Part G: Following TCP Streams

Step-1: Right-click on a TCP packet in the Packet List pane.
Step-2: Select **Follow -> TCP Stream**.
Step-3: A new window opens showing the entire TCP conversation in both directions.
Step-4: You can view the data in ASCII, HEX, or other formats.
Step-5: This is useful for reconstructing HTTP requests, FTP transfers, and other application-layer traffic.

Part H: Using Colouring Rules

Step-1: Go to **View -> Coloring Rules**.
Step-2: Wireshark has default color rules for common protocols (e.g., TCP is blue, UDP is light blue, HTTP is green).
Step-3: To add a custom coloring rule:

- Click **New**.
- Enter a name and a display filter expression.
- Choose a foreground and background color.
- Click **OK**.

EXAMPLES:

Example 1: Capturing HTTP Traffic Only

Capture Filter:

```
port 80
```

Display Filter:

```
http
```

Example 2: Capturing Traffic to/from a Specific IP

Capture Filter:

```
host 192.168.1.10
```

Display Filter:

```
ip.addr == 192.168.1.10
```

Example 3: Following a TCP Stream (HTTP GET Request)

1. Capture traffic while browsing a website.
2. In the Packet List, find an HTTP GET packet.
3. Right-click -> **Follow -> TCP Stream**.
4. The stream shows the full HTTP request and response.

Example 4: Command-Line Capture

```
wireshark -i eth0 -f "tcp port 80" -w http_traffic.pcap
```

CONSTRAINTS:

- Wireshark requires administrative/root privileges to capture packets.
- In switched networks, only the capturing computer's own traffic is visible unless port mirroring is enabled.
- Capturing all traffic on a busy network can generate very large files - use capture filters to reduce data.
- Analyzing encrypted traffic (HTTPS, SSH, etc.) requires decryption keys (which are not available in typical scenarios).

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/satishmettela04-art/exp13-cn.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: satishmettela04-art
Repo Name: exp13-cn
Default Branch: main
Fetched Files:
```

Performance Summary

Solution evaluation complete

100	0%	0ms
POINTS	RATE	TIME

ACCURACY **0/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 9:58:50 PM**

Experiment-14

AIM:

The aim of this experiment is to learn how to run Nmap scans for network discovery, port scanning, OS detection, and service enumeration using both the command-line interface and the Zenmap GUI.

PROBLEM STATEMENT:

1. Download and install Nmap (with Zenmap GUI) on Windows/Linux/macOS.
2. Use Zenmap to perform different types of scans:
   - **Ping Scan** - detect live hosts
   - **Quick Scan** - scan common ports
   - **Regular Scan** - standard port scan
   - **Intense Scan** - comprehensive scan with OS detection, version detection, and script scanning
3. Use the command-line version of Nmap to perform scans with custom parameters.
4. Interpret the scan results: open ports, services, OS, and traceroute.
5. Document the findings and analyse the network topology.

INPUT:

- Target IP address, domain name, or network range (e.g., `192.168.1.0/24`, `example.com`)

OUTPUT:

- Scan results including:
  - Online/offline status of hosts
  - Open ports and associated services
  - Detected operating system
  - Traceroute information
  - Script scan results (if applicable)

Example Output (Nmap Command Line):

```
Starting Nmap 7.94 ( https://nmap.org ) at 2026-01-15 10:30 IST
Nmap scan report for 192.168.1.1
Host is up (0.0050s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE
80/tcp   open  http
443/tcp  open  https
22/tcp   open  ssh
MAC Address: 00:1A:2B:3C:4D:5E (Cisco Systems)

Nmap done: 1 IP address (1 host up) scanned in 2.34 seconds
```

Example Output (Zenmap):

```
Intense Scan Report for 192.168.1.10
Host is up (0.0010s latency).
PORT     STATE SERVICE    VERSION
22/tcp   open  ssh        OpenSSH 8.2p1
80/tcp   open  http       Apache httpd 2.4.41
443/tcp  open  https      Apache httpd 2.4.41
3306/tcp open  mysql      MySQL 8.0.28
Device type: general purpose
Running: Linux 5.x
OS CPE: cpe:/o:linux:linux_kernel:5
```

ALGORITHM / PROCEDURE:

Part A: Download and Install Nmap

Step-1: Open a web browser and navigate to the official Nmap website: https://nmap.org/
Step-2: Click on the **Download** button and select the appropriate installer for your operating system.
Step-3: Windows:

- Download the Windows installer (e.g., `nmap-7.94-setup.exe`).
- Run the installer and follow the wizard.
- Keep all components checked (Nmap, Zenmap, Nping, etc.).
- Complete the installation.

Step-4: Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install nmap zenmap
```

Step-5: macOS:

- Download the macOS installer `.dmg` file.
- Open the file and drag Nmap to the Applications folder.

Part B: Using Zenmap (GUI)

Step-1: Launch Zenmap from the Start Menu (Windows), Applications menu (Linux), or Applications folder (macOS).
Step-2: In the **Target** field, enter the target to scan (e.g., `192.168.1.1`, `example.com`, `192.168.1.0/24`).
Step-3: In the **Profile** dropdown, select a scan profile:

Profile	Description
Intense scan	Comprehensive scan: OS detection, version detection, script scanning, traceroute (aggressive timing)
Ping scan	Only detects live hosts (no port scanning)
Quick scan	Faster scan with aggressive timing and common ports
Regular scan	Standard Nmap scan (default options)

Step-4: Click the **Scan** button to start scanning.
Step-5: Observe the progress in the **Nmap Output** tab.
Step-6: Once the scan completes, explore the results in different tabs:

- **Ports/Hosts** - open ports and services
- **Topology** - traceroute to the target
- **Host Details** - summary of the target
- **Scans** - command history

Part C: Using Nmap from Command Line

Step-1: Open Command Prompt (Windows) or Terminal (Linux/macOS).
Step-2: Basic scan syntax:

```
nmap <target>
```

Step-3: Common scan commands:

Command	Description
`nmap -sn 192.168.1.0/24`	Ping scan (discover live hosts)
`nmap -sS 192.168.1.10`	TCP SYN stealth scan
`nmap -sT 192.168.1.10`	TCP Connect scan
`nmap -sU 192.168.1.10`	UDP scan
`nmap -sV 192.168.1.10`	Version detection
`nmap -O 192.168.1.10`	OS detection

`nmap -A 192.168.1.10`	Aggressive scan (OS, version, scripts, traceroute)
`nmap -p 22,80,443 192.168.1.10`	Scan specific ports
`nmap -p- 192.168.1.10`	Scan all ports (1-65535)
`nmap --script=vuln 192.168.1.10`	Vulnerability scan using NSE scripts

Step-4: View help:

```
nmap -h
```

Part D: Interpreting Results

Step-1: In the **Nmap Output** tab, review the scan summary:

- Number of hosts up/down
- Open ports and services
- OS detection results

Step-2: In the **Ports/Hosts** tab:

- List all open ports
- View the service running on each port

Step-3: In the **Topology** tab:

- View the traceroute path to the target
- See the number of hops

Step-4: In the **Host Details** tab:

- View a summary of the target:
  - IP address
  - Hostname
  - Operating system
  - Open ports count

Part E: Saving Scan Results

Step-1: In Zenmap, go to **File -> Save Scan** to save results in `.xml` or `.txt` format.
Step-2: In the command line, use output options:

```
nmap -oN output.txt <target>    # Normal output
nmap -oX output.xml <target>    # XML output
nmap -oG output.gnmap <target>  # Grepable output
```

EXAMPLES:

Example 1: Ping Scan

Command:

```
nmap -sn 192.168.1.0/24
```

Output:

```
Starting Nmap 7.94 ( https://nmap.org ) at 2026-01-15 10:30 IST
Nmap scan report for 192.168.1.1
Host is up (0.0010s latency).
Nmap scan report for 192.168.1.10
Host is up (0.0020s latency).
Nmap scan report for 192.168.1.20
Host is up (0.0015s latency).
```

```
...
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.34 seconds
```

Example 2: Aggressive Scan (Command Line)

Command:

```
nmap -A 192.168.1.10
```

Output:

```
Starting Nmap 7.94 ( https://nmap.org ) at 2026-01-15 10:35 IST
Nmap scan report for 192.168.1.10
Host is up (0.0010s latency).
Not shown: 993 closed tcp ports (reset)
PORT     STATE SERVICE    VERSION
22/tcp   open  ssh        OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
80/tcp   open  http       Apache httpd 2.4.41 ((Ubuntu))
443/tcp  open  https      Apache httpd 2.4.41 ((Ubuntu))
3306/tcp open  mysql      MySQL 8.0.28
Device type: general purpose
Running: Linux 5.x
OS CPE: cpe:/o:linux:linux_kernel:5
OS details: Linux 5.0 - 5.15
Network Distance: 1 hop
```

Example 3: Intense Scan (Zenmap)

Target: `192.168.1.1`
Profile: Intense scan
Result: Open ports: 22 (SSH), 80 (HTTP), 443 (HTTPS). OS: Linux 4.4.

CONSTRAINTS:

- Nmap requires administrative/root privileges for certain scan types (e.g., SYN scan, OS detection).
- Scanning networks without permission may be illegal - only scan networks you own or have explicit permission to scan.
- Some firewalls may block Nmap scans, resulting in inaccurate results.

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/satishmettela04-art/exp-13-cn.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: satishmettela04-art
Repo Name: exp-13-cn
Default Branch: main
Fetched Files:
```

Performance Summary

Solution evaluation complete

100	0%	0ms
POINTS	RATE	TIME

ACCURACY **0/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 10:02:46 PM**

Experiment-15

AIM:

The aim of this experiment is to perform Operating System (OS) detection and service version detection on a remote host using Nmap's `-O` and `-sV` flags, and to interpret the results to identify the target's OS, open ports, and running services.

PROBLEM STATEMENT:

1. Use Nmap with the `-O` (OS detection) and `-sV` (version detection) flags to scan a target IP address.
2. Analyse the scan output to identify:
   - Open ports and their services
   - Service versions
   - Operating system of the target
   - Network distance (hops)
3. Document the findings and confirm the OS.

INPUT:

- Target IP address (e.g., `x.x.x.x`)

OUTPUT:

- List of open ports with service names and versions
- Operating system guesses with confidence percentages
- Network distance (number of hops)
- Service information summary

Example Output:

```
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
2701/tcp open  cmrcservice   Microsoft Configuration Manager Remote Control service
(CmRcService.exe)
3306/tcp open  mysql         MySQL 5.1.60-community-log
3389/tcp open  ms-wbt-server Microsoft Terminal Services

Aggressive OS guesses: Microsoft Windows Server 2016 (93%),
Microsoft Windows Server 2008 R2 SP1 (92%),
Microsoft Windows Server 2012 R2 (91%),
...

Network Distance: 18 hops
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

ALGORITHM / PROCEDURE:

Step-1: Open a terminal or command prompt.
Step-2: Run the Nmap command with OS detection and version detection:

```
nmap -O -sV <target_ip>
```

Step-3: Wait for the scan to complete (may take several seconds to minutes).
Step-4: Observe the output:

- **Port/Service Information**: Lists open ports, state, service name, and version
- **OS Guesses**: Shows possible operating systems with confidence levels
- **Network Distance**: Number of hops between the scanning host and target
- **Service Info**: Summary of the target's operating system

Step-5: Interpret the results:

- If the target is a Windows system, look for ports like 135 (RPC), 139/445 (NetBIOS/SMB), 3389 (RDP)
- If the target is a Linux system, look for ports like 22 (SSH), 80 (HTTP), 443 (HTTPS), 3306 (MySQL)

Step-6: Document the findings and confirm the OS based on the open ports, services, and OS guesses.

EXAMPLES:

Example 1: Windows Server Scan

Command:

```
nmap -O -sV 192.168.1.100
```

Output:

```
PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
3389/tcp open  ms-wbt-server Microsoft Terminal Services

Aggressive OS guesses: Microsoft Windows Server 2019 (95%),
Microsoft Windows Server 2016 (93%),
...

Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

Interpretation: The target is a Windows Server with RPC, NetBIOS, SMB, and RDP services.

Example 2: Linux Server Scan

Command:

```
nmap -O -sV 192.168.1.101
```

Output:

```
PORT     STATE SERVICE    VERSION
22/tcp   open  ssh        OpenSSH 8.2p1 Ubuntu 4ubuntu0.5
80/tcp   open  http       Apache httpd 2.4.41 ((Ubuntu))
443/tcp  open  https      Apache httpd 2.4.41 ((Ubuntu))
3306/tcp open  mysql      MySQL 8.0.28

Aggressive OS guesses: Linux 5.0 - 5.15 (95%),
Ubuntu Linux 20.04 (94%),
...

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel:5
```

Interpretation: The target is a Linux server (likely Ubuntu) with SSH, HTTP,HTTPS, and MySQL services.

CONSTRAINTS:

- Nmap requires administrative/root privileges for OS detection (`-O`) on some systems.
- Scanning networks without permission may be illegal - only scan networks you own or have explicit permission to scan.
- Firewalls and network filters may affect scan results, reducing accuracy.
- OS detection may not be 100% accurate; it provides the most likely OS based on fingerprinting.

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/satishmettela04-art/exp15-cn.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: satishmettela04-art
Repo Name: exp15-cn
Default Branch: main
Fetched Files:
```

Performance Summary

Solution evaluation complete

100	0%	0ms
POINTS	RATE	TIME

ACCURACY **0/1 Passed**

Student Name: METTELA SATISH **Date: Oct 12, 2026, 10:04:33 PM**

Experiment-16

AIM:

The aim of this experiment is to use the NS2 Simulator to:

1. Understand the basics of NS2
2. Simulate and find the number of packets dropped
3. Simulate and find the number of packets dropped by TCP/UDP
4. Simulate and find the number of packets dropped due to congestion
5. Simulate and compare data rate and throughput

PROBLEM STATEMENT:

Write TCL scripts for NS2 to perform the following simulations:

1. **Introduction to NS2** - Understand the basic architecture and components
2. **Packet Drop Simulation** - Count total number of packets dropped in a network
3. **TCP/UDP Packet Drop** - Differentiate between packets dropped by TCP and UDP traffic
4. **Congestion-based Packet Drop** - Analyze packet drops specifically caused by congestion
5. **Data Rate & Throughput Comparison** - Compare data rate and throughput for different configurations

INPUT:

- NS2 simulator installed on Linux/Windows/Mac
- TCL scripts for each simulation

OUTPUT:

- Trace files (`.tr`) containing packet-level information
- Nam files (`.nam`) for visualization
- Analysis results showing packet drop counts and throughput values

ALGORITHM / PROCEDURE:

Part (i): NS2 Simulator - Introduction

Step-1: Install NS2 on your system (Linux recommended).
Step-2: Understand the basic architecture:
- C++ backend for core functionality
- OTcl frontend for simulation scripts
- Trace files for output analysis

Step-3: Learn basic TCL commands for NS2:
- Creating simulator object
- Creating nodes and links
- Attaching agents
- Scheduling events

Basic NS2 Script Template:

```
# Create simulator object
set ns [new Simulator]

# Open trace files
set tracefile [open output.tr w]
$ns trace-all $tracefile

# Define finish procedure
proc finish {} {
    global ns tracefile
    $ns flush-trace
    close $tracefile
```

```
    exit 0
}

# Create nodes
set n0 [$ns node]
set n1 [$ns node]

# Create link
$ns duplex-link $n0 $n1 10Mb 10ms DropTail

# Schedule events
$ns at 1.0 "finish"

# Run simulation
$ns run
```

Part (ii): Simulate to Find the Number of Packets Dropped

Step-1: Create a network topology with a bottleneck link.
Step-2: Set a limited queue size to force packet drops.
Step-3: Generate traffic using FTP/CBR sources.
Step-4: Run the simulation and analyze the trace file to count dropped packets.

```
# Create simulator object
set ns [new Simulator]

# Open trace file
set tracefile [open drop.tr w]
$ns trace-all $tracefile

# Create 4 nodes (n0-n3)
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

# Create links (n0-n2 and n1-n2 with 10Mbps, n2-n3 with 1Mbps bottleneck)
$ns duplex-link $n0 $n2 10Mb 10ms DropTail
$ns duplex-link $n1 $n2 10Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail

# Set queue limit on bottleneck link
$ns queue-limit $n2 $n3 10

# Create TCP agent and attach to n0
set tcp [new Agent/TCP/Reno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink

# Create FTP traffic
set ftp [new Application/FTP]
$ftp attach-agent $tcp
```

```
# Schedule events
$ns at 0.5 "$ftp start"
$ns at 4.5 "$ftp stop"
$ns at 5.0 "finish"

$ns run
```

Part (iii): Simulate to Find the Number of Packets Dropped by TCP/UDP

Step-1: Create a topology with TCP and UDP sources sharing a bottleneck link.
Step-2: Count packets dropped from TCP and UDP flows separately.
Step-3: Compare the drop rates.

```
# Create simulator object
set ns [new Simulator]

# Open trace file
set tracefile [open tcpudp.tr w]
$ns trace-all $tracefile

# Create nodes
set n0 [$ns node]   # TCP source
set n1 [$ns node]   # UDP source
set n2 [$ns node]   # Router
set n3 [$ns node]   # Destination

# Create links
$ns duplex-link $n0 $n2 10Mb 10ms DropTail
$ns duplex-link $n1 $n2 10Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
$ns queue-limit $n2 $n3 20

# TCP setup
set tcp [new Agent/TCP/Reno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]
$ftp attach-agent $tcp

# UDP setup
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
set cbr [new Application/Traffic/CBR]
$cbr set packetSize_ 500
$cbr set interval_ 0.01
$cbr attach-agent $udp

# Schedule events
$ns at 0.5 "$ftp start"
```

```
$ns at 0.5 "$cbr start"
$ns at 4.5 "$ftp stop"
$ns at 4.5 "$cbr stop"
$ns at 5.0 "finish"

$ns run
```

Analyzing TCP vs UDP drops: In the trace file, packets dropped by TCP and UDP can be identified. UDP traffic tends to drop packets without adjusting its rate, while TCP responds to drops by reducing its sending rate.

Part (iv): Simulate to Find the Number of Packets Dropped due to Congestion

Step-1: Create a network with multiple sources sending to a bottleneck link.
Step-2: The bottleneck link has a limited queue size.
Step-3: When the queue overflows, packets are dropped due to congestion.
Step-4: Analyze the trace file to count congestion-related drops.

```
# Create simulator object
set ns [new Simulator]

# Open trace file
set tracefile [open congestion.tr w]
$ns trace-all $tracefile

# Create 5 nodes (n0-n4)
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]

# Create links
$ns duplex-link $n0 $n4 10Mb 10ms DropTail
$ns duplex-link $n1 $n4 10Mb 10ms DropTail
$ns duplex-link $n2 $n4 10Mb 10ms DropTail
$ns duplex-link $n4 $n3 1Mb 10ms DropTail

# Set small queue on bottleneck to cause congestion
$ns queue-limit $n4 $n3 10

# Create TCP sources for n0, n1, n2
for {set i 0} {$i < 3} {incr i} {
    set tcp($i) [new Agent/TCP/Reno]
    $ns attach-agent $n$i $tcp($i)
    set sink($i) [new Agent/TCPSink]
    $ns attach-agent $n3 $sink($i)
    $ns connect $tcp($i) $sink($i)
    set ftp($i) [new Application/FTP]
    $ftp($i) attach-agent $tcp($i)
    $ns at 0.5 "$ftp($i) start"
}

$ns at 4.5 "finish"
$ns run
```

Part (v): Simulate to Compare Data Rate & Throughput

Step-1: Run simulations with different data rates (e.g., 1Mbps, 5Mbps, 10Mbps).
Step-2: Calculate throughput for each configuration.
Step-3: Compare data rate and throughput to observe the relationship.

```
# Function to run simulation with given data rate
proc run_simulation {rate} {
    set ns [new Simulator]
    set tracefile [open throughput_${rate}.tr w]
    $ns trace-all $tracefile

    # Create topology
    set n0 [$ns node]
    set n1 [$ns node]
    $ns duplex-link $n0 $n1 ${rate}Mb 10ms DropTail

    # TCP setup
    set tcp [new Agent/TCP/Reno]
    $ns attach-agent $n0 $tcp
    set sink [new Agent/TCPSink]
    $ns attach-agent $n1 $sink
    $ns connect $tcp $sink
    set ftp [new Application/FTP]
    $ftp attach-agent $tcp

    $ns at 0.5 "$ftp start"
    $ns at 4.5 "$ftp stop"
    $ns at 5.0 "finish"
    $ns run
}

# Run for different rates
run_simulation 1
run_simulation 5
run_simulation 10
```

EXAMPLES:

Example 1: Packet Drop Analysis

The trace file (`drop.tr`) contains lines with `D` (drop) indicating dropped packets. Counting these lines gives the total number of packets dropped.

Example 2: Throughput Calculation

Throughput is calculated as:

```
Throughput = (Total bytes received) / (Simulation time)
```

Higher data rates generally result in higher throughput, but factors like congestion and packet loss can reduce effective throughput.

CONSTRAINTS:

- NS2 must be installed and properly configured.
- Simulations must be run with appropriate queue sizes to observe packet drops.

- Trace files must be analyzed correctly to extract meaningful metrics.
- The simulation time and traffic patterns must be consistent for fair comparisons.

STUDENT SUBMISSION / CODE:

```
GitHub Repository: https://github.com/satishmettela04-art/exp16-cn.git
Notes:
```

OUTPUT & ANALYSIS:

```
Valid Repository: Yes
Owner: satishmettela04-art
Repo Name: exp16-cn
Default Branch: main
Fetched Files:
```

Performance Summary

Solution evaluation complete

100	0%	0ms
POINTS	RATE	TIME

ACCURACY **0/1 Passed**